

Biodiversity assignment

Before you start

Please make sure that

- 1) You are working on the correct assignment, you have been randomly assigned one of two assignments you cannot choose which one to answer.
- 2) You have read and understood the power point slides on Plagiarism and Referencing
- 3) You understand the key programming concepts such as variables, loops, functions, lists and files.
- 4) Generative AI tools **cannot** be used in any way in the assessment

Introduction

Understanding the variety and frequency of species in a given area (biodiversity) is a key challenge in biology. Climate change, invasive species, habitat destruction and overexploitation are all causes of biodiversity loss, which is accelerating at an unprecedented rate. Understanding changes in biodiversity is fundamental to assessing our impact on the environment, identifying species and habitats which are most at risk and evaluating conservation efforts.

Biodiversity assessments must be data driven to ensure they are valid. The Global Biodiversity Information Facility (www.gbif.org) is a web site designed to provide access to biodiversity data from numerous studies, currently it holds nearly 2 billion records, from over 65,000 datasets.

This assignment will provide an introduction to working with biodiversity data and estimating the biodiversity for a number of locations within the UK and how geographical data can be visualised.

Data

A data set of mammals has been created from GBIF, Mammal.txt contains 125,000 sightings of mammals. Each entry consists of three fields, a species name, a latitude and a longitude, the fields are separated using tabs. A data set of birds (Birds.txt) is also included for testing purposes, it is in the same file format.

Details

A Python script "Biodiversity.py" contains two functions *CalculateDistance* which calculates the distance between two locations. *CalculateDistance* takes four variables, the latitude and longitude of the first location and the latitude and longitude of the second location, it returns the distance in km between the two locations. The second function, *LineToList*, converts a line of the input file to a list of three elements. Breaking the line on tabs, the function takes a line and returns a list.

The assignment is broken down into three sections, which get progressively harder and require more independent development.

Assignment

1) Find the number of animals at a given distance from a specified location (40%)

Write a function *LocationCount* which returns the number of animals found within a specified distance of a given location, the function should take four parameters, a file name containing the data, a distance in kilometres, and the latitude and longitude of the location. It should return the number of animals found within the specified distance of the location.

Rough pseudo code for the function is given below

- i) Load the data from the file into a list
- ii) For every line in the list
 - Use *LineToList* function to split each line into its three components, species name, latitude and longitude
 - Use the *CalculateDistance* function to determine if the animal is within the specified distance of the location and keep a count of the total number who are.
- iii) Return the number of animals which were within the specified distance of the location

Your script should call *LocationCount* with the data file "Mammal.txt", a distance of 10.0, a latitude of 54.988056 and longitude of -1.619444 (Newcastle). The result should be printed to the screen.

2) Visualise the data using KML files (30%)

Google maps allows you to visualise geographical data by plotting features on the map. These features are specified in a kml file. Information about creating kml files and viewing them with google maps can be found in the handbook and lecture notes.

Write a function *PrintLocation* which takes four parameters, a file name containing animal location data, a distance in kilometres, and the latitude and longitude of the location. It should output, to a file called "output.kml", in kml format all animals found within the distance of the specified location. This function will be similar to *LocationCount* in its logic.

Your script should call *PrintLocation* with the data file "Mammal.txt", a distance of 15.0, a latitude of 51.452884 and longitude of -0.973906 (Reading).

3) Calculate the biodiversity in a specific region (30%)

Write a function *BiodiversityCount* which takes four parameters, a file name containing animal location data, a distance in kilometres, and the latitude and longitude of the location. The function should return the number of unique species found in the area.

Your script should call *BiodiversityCount* with the data file “Mammal.txt”, a distance of 25.0, a latitude of 51.508129 and longitude of -0.128005 (London). The result should be printed to the screen.

Deliverables

Your Python script implementing the program defined above, the code should be well written, clear, commented, and correct. The script should run without errors or warnings. All three part should be included in one python file.

Test information

Data File	Distance	Lat	Long	No Animals	KML File	No Species
Mammal.txt	20.0	50.261667	-5.043333	389	Test01.kml	42
Mammal.txt	5.0	51.75	-1.25	95	Test02.kml	30
Mammal.txt	15.0	53.8	-1.583333	356	Test03.kml	33
Birds.txt	7.5	52.966667	-1.166667	1173	Test04.kml	38
Birds.txt	5.0	51.452884	-0.973906	737	Test05.kml	31

Hints and Tips

- 1) Read the assignment thoroughly and follow the instructions carefully.
- 2) This is not a typing exercise, my solution was around 65 lines of uncommented code, if your solution is significantly longer, you are most likely on the wrong lines.
- 3) Use the test data and results provided to ensure your program is working correctly, come up with some sanity checks of your own.
- 4) Refer to the textbook, previous code, examples and the web when stuck.
- 5) The program may take a few minutes to run.
- 6) Parts 1, 2 and 3 are very similar, once you have the logic for part 1 correct it can be adapted for parts 2 and 3.
- 7) Make sure your work is backed up and archived when making changes.